

DOCUMENTACIÓN TÉCNICA

Proyecto ISI SmartWatt

1. Introducción

1.1 Descripción general

ISI SmartWatt es una aplicación orientada a la monitorización, gestión y visualización de información energética y meteorológica mediante una arquitectura web basada en tecnologías modernas de desarrollo backend y frontend.

El proyecto ha sido desarrollado siguiendo una estructura modular que separa claramente las responsabilidades entre la lógica de negocio, la interfaz de usuario y la configuración de despliegue. Además, incorpora automatización mediante contenedores Docker, configuración mediante variables de entorno y automatización de tareas a través de Makefile.

La finalidad principal del sistema es proporcionar una plataforma capaz de gestionar información relacionada con consumo energético y condiciones climáticas, integrando datos externos obtenidos mediante APIs.

1.2 Objetivos del proyecto

Los objetivos principales del sistema son:

- Implementar una arquitectura web modular y escalable.
 - Separar frontend y backend siguiendo buenas prácticas.
 - Permitir la integración con APIs externas.
 - Facilitar el despliegue mediante Docker.
 - Gestionar dependencias y automatizar tareas de ejecución.
 - Aplicar una estructura profesional de desarrollo software.
 - Permitir futuras ampliaciones del sistema.
-

1.3 Alcance

El proyecto cubre:

- Desarrollo de backend en Python.
- Interfaz frontend para interacción con usuarios.
- Gestión de configuración mediante variables de entorno.
- Integración con OpenWeather API.
- Persistencia de datos mediante MySQL.
- Automatización del entorno de desarrollo.
- Contenerización de servicios.
- Pruebas automatizadas.

No se incluyen en esta versión:

- Sistemas avanzados de autenticación.
 - Gestión multiusuario avanzada.
 - Escalado distribuido.
 - Balanceadores de carga.
 - Sistemas de monitorización empresarial.
-

2. Arquitectura del sistema

2.1 Arquitectura general

El proyecto sigue una arquitectura cliente-servidor distribuida.

La aplicación se divide en los siguientes componentes:

1. Frontend
2. Backend
3. Base de datos MySQL
4. Servicios externos (OpenWeather API)
5. Infraestructura Docker y Nginx

La comunicación entre frontend y backend se realiza mediante peticiones HTTP.

2.2 Componentes principales

Frontend

Responsable de:

- Mostrar información al usuario.

- Gestionar interacción visual.
- Consumir endpoints del backend.
- Presentar datos meteorológicos y energéticos.

Tecnologías utilizadas:

- HTML
 - CSS
 - JavaScript
-

Backend

Responsable de:

- Procesamiento lógico.
- Integración con APIs externas.
- Gestión de datos.
- Comunicación con base de datos.
- Exposición de endpoints.

Tecnologías utilizadas:

- Python
 - Flask
-

Base de datos

La aplicación utiliza MySQL para persistencia de información.

Se incluyen archivos de definición de esquema y modelo de datos:

- `schema_mysql.sql`
- `MODELO_DATOS_MYSQL.md`

La base de datos permite:

- Almacenar información energética.
 - Persistir registros meteorológicos.
 - Gestionar relaciones entre entidades.
-

Docker

El proyecto incorpora contenerización mediante:

- Dockerfile
- docker-compose.yml

Esto permite:

- Despliegues reproducibles.
 - Aislamiento de dependencias.
 - Facilidad de instalación.
 - Compatibilidad multiplataforma.
-

Nginx

Nginx actúa como servidor web y proxy inverso.

Funciones principales:

- Gestión de tráfico HTTP.
 - Servido de archivos estáticos.
 - Proxy hacia backend.
 - Optimización de acceso.
-

3. Tecnologías utilizadas

Tecnología	Uso principal
Python	Backend
HTML/CSS/JavaScript	Frontend
MySQL	Base de datos
Docker	Contenerización
Docker Compose	Orquestación
Nginx	Proxy inverso
Makefile	Automatización
Pytest	Testing
OpenWeather API	Datos meteorológicos

4. Estructura del proyecto

4.1 Organización del repositorio

isi-smartwatt/

```
|
|— backend/
|— frontend/
|— test/
|— app.py
|— docker-compose.yml
|— Dockerfile
|— nginx.conf
|— requirements.txt
|— Makefile
|— schema_mysql.sql
|— MODELO_DATOS_MYSQL.md
|— interfaces_estructuras.md
|— README.md
```

4.2 Descripción de directorios

backend/

Contiene la lógica principal del sistema:

- Procesamiento de datos.
 - Gestión de APIs.
 - Comunicación con base de datos.
 - Endpoints del sistema.
-

frontend/

Incluye:

- Interfaz gráfica.
 - Recursos visuales.
 - Scripts del cliente.
 - Diseño de experiencia de usuario.
-

test/

Directorio destinado a:

- Pruebas automatizadas.
 - Validación funcional.
 - Verificación del comportamiento del sistema.
-

5. Funcionamiento del sistema

5.1 Flujo general de operación

1. El usuario accede al frontend.
 2. El frontend realiza solicitudes HTTP al backend.
 3. El backend procesa la petición.
 4. Si es necesario, consulta APIs externas o base de datos.
 5. Los datos se procesan y devuelven al frontend.
 6. El frontend muestra la información al usuario.
-

5.2 Integración con OpenWeather

La aplicación utiliza OpenWeather API para obtener datos meteorológicos.

Proceso:

1. El sistema envía peticiones HTTP a OpenWeather.
 2. Se autentica mediante API Key.
 3. Se reciben datos meteorológicos.
 4. Los datos son procesados y almacenados.
 5. Se presentan al usuario final.
-

5.3 Variables de entorno

El proyecto utiliza variables de entorno para evitar exponer información sensible.

Ejemplo:

`OPENWEATHER_API_KEY=tu_api_key`

Ventajas:

- Mayor seguridad.

- Configuración flexible.
- Separación entre código y configuración.
- Facilita despliegues.

5.4 Sistema de Inteligencia Artificial

El proyecto incorpora funcionalidades de Inteligencia Artificial mediante el uso de modelos de lenguaje ejecutados localmente con Ollama.

La IA se utiliza para mejorar la capacidad de análisis y procesamiento de información energética, permitiendo futuras ampliaciones relacionadas con recomendaciones inteligentes y automatización de decisiones.

- Ollama

Ollama se utiliza como entorno de ejecución local para modelos de lenguaje (LLMs).

Ventajas principales:

- Ejecución local sin depender de servicios externos.
- Mayor privacidad de los datos.
- Facilidad de integración con Python.
- Bajo consumo de recursos.

- Gemma3:1b

El sistema utiliza el modelo Gemma3:1b como modelo base de IA.

Características:

- Modelo ligero optimizado para ejecución local.
- Buen equilibrio entre rendimiento y consumo.
- Integración sencilla con aplicaciones backend.
- Capacidad de procesamiento de lenguaje natural.

Posibles usos dentro del sistema:

- Generación de recomendaciones energéticas.
- Interpretación de consultas del usuario.
- Asistencia inteligente.
- Automatización de respuestas.

6. Base de datos

6.1 Modelo de datos

El proyecto incorpora documentación específica del modelo de datos en:

- `MODELO_DATOS_MYSQL.md`

Y el esquema SQL en:

- `schema_mysql.sql`

La base de datos está diseñada siguiendo principios relacionales.

6.2 Características principales

La base de datos permite:

- Persistencia estructurada.
 - Relaciones entre entidades.
 - Consultas eficientes.
 - Escalabilidad futura.
-

7. Automatización y despliegue

7.1 Uso de Makefile

El proyecto utiliza un Makefile para simplificar tareas comunes.

make start

Realiza:

- Creación de entorno virtual.
 - Instalación de dependencias.
 - Preparación inicial del sistema.
-

make run

make run

make test

make test

make clean

make clean

7.2 Docker Compose

Docker Compose permite levantar todos los servicios del sistema de manera centralizada.

Ventajas:

- Simplificación del despliegue.
 - Reproducibilidad.
 - Facilidad de configuración.
 - Gestión centralizada de contenedores.
-

7.3 Nginx

El servidor Nginx mejora el rendimiento del sistema mediante:

- Proxy inverso.
- Gestión de conexiones.
- Distribución de peticiones.
- Servicio de contenido estático.

7.4 Monitorización y métricas

El proyecto incorpora mecanismos básicos de monitorización mediante comandos automatizados incluidos en el Makefile.

- make logs

El comando: ``bash make logs`` permite visualizar los registros de funcionamiento de la aplicación y los contenedores Docker.

Esto facilita:

- Supervisión del sistema.
- Detección de errores.
- Seguimiento de eventos.
- Análisis de comportamiento de la aplicación.

- Obtención de métricas básicas de funcionamiento.
-

8. Testing

8.1 Estrategia de pruebas

El proyecto incorpora pruebas automatizadas mediante Pytest.

Objetivos:

- Validar funcionamiento.
 - Detectar errores.
 - Garantizar estabilidad.
 - Facilitar mantenimiento.
-

8.2 Tipos de pruebas

Las pruebas pueden incluir:

- Tests unitarios.
- Tests funcionales.
- Tests de integración.

8.2 Cobertura de pruebas

Actualmente el proyecto incorpora cinco pruebas automatizadas destinadas a validar diferentes componentes del sistema.

Estas pruebas permiten:

- Verificar el correcto funcionamiento del backend.
- Validar la comunicación entre componentes.
- Detectar errores durante el desarrollo.
- Garantizar estabilidad básica del sistema.

La utilización de Pytest facilita la ampliación futura del sistema de testing conforme evolucione el proyecto.

9. Seguridad

9.1 Gestión de credenciales

Las credenciales se gestionan mediante variables de entorno.

Esto evita:

- Exposición de claves.
- Inclusión de datos sensibles en el repositorio.

9.2 Buenas prácticas aplicadas

El proyecto aplica:

- Separación de responsabilidades.
- Modularidad.
- Configuración desacoplada.
- Uso de contenedores.
- Automatización.

9.3 Archivo AGENT.md

El proyecto incorpora un archivo `AGENT.md` destinado a documentar el comportamiento y funcionamiento de los agentes inteligentes integrados en el sistema.

Este archivo permite:

- Definir responsabilidades de los agentes.
- Documentar flujos de interacción.
- Facilitar mantenimiento y ampliaciones.
- Centralizar la información relacionada con la IA.

La utilización de este tipo de documentación mejora la organización interna del sistema y facilita futuras integraciones basadas en agentes inteligentes.

10. Posibles mejoras futuras

El sistema puede evolucionar incorporando:

- Autenticación JWT.
 - Panel de administración.
 - Sistema multiusuario.
 - Gráficas avanzadas.
 - Monitorización en tiempo real.
 - Integración IoT.
 - Escalabilidad cloud.
 - Caché distribuida.
 - CI/CD.
 - Monitorización avanzada.
-

12. Conclusiones

ISI SmartWatt representa un proyecto desarrollado siguiendo principios modernos de ingeniería del software.

La arquitectura modular, el uso de contenedores, la automatización mediante Makefile y la separación entre frontend y backend permiten mantener una estructura profesional y escalable.

La integración con APIs externas y la utilización de tecnologías ampliamente utilizadas en entornos reales convierten el proyecto en una solución técnicamente sólida y adecuada para entornos académicos y de desarrollo profesional.

Además, la organización del repositorio y la orientación a buenas prácticas facilitan futuras ampliaciones y mantenimiento evolutivo del sistema.

13. Anexos

13.1 Archivos principales

Archivo	Función
app.py	Punto de entrada principal

Dockerfile	Construcción de contenedor
docker-compose.yml	Orquestación de servicios
requirements.txt	Dependencias Python
nginx.conf	Configuración Nginx
schema_mysql.sql	Esquema de base de datos
Makefile	Automatización